Program, process, and PCB

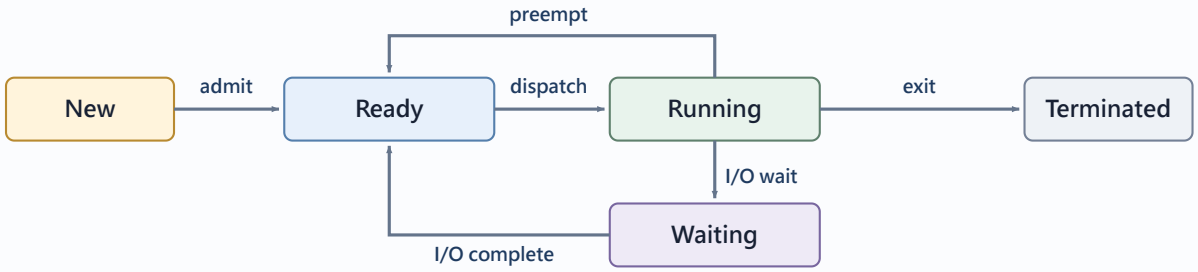
A program describes executable instructions/data. A process is one identified execution with resources and state. Two launches of a counter image, P100 and P101, can share read-only physical text while retaining private globals, heap, stack, registers, and descriptor tables.

State	P100	P101	Sharing condition
PID, PC, r1	100,0x40,7	101,0x120,3	Execution context is independent
Private counter at VA 0x5000	0→1	0→1	No shared mapping, no combined increment
fd3	OA→file A	OB→file B	Same number, different object
Code	Read-only image text	Same text	Frames can be shared without sharing execution

The PCB tracks identity, state, PC/registers, scheduling, and memory/I/O references. It can reference page tables and open descriptions without embedding the entire heap. Multithreaded execution separates additional thread context in Chapter 4.

Five states and their causes

Event completion makes Ready; dispatch supplies the CPU

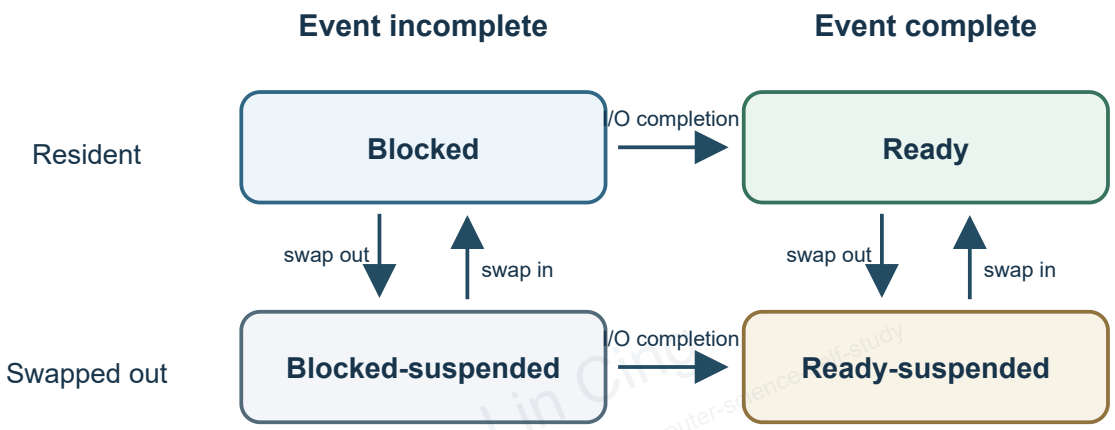


Arrows: legal state transitions. One CPU runs at most one process. Mode changes need not leave Running.

Assume one CPU and one execution stream per process. At most one process is running. Ready means executable but awaiting CPU; waiting means an event is missing. This abstraction does not enumerate every OS substate or imply that terminated metadata has disappeared.

Admission is new→ready; dispatch ready→running; preemption running→ready; blocking running→waiting; event completion waiting→ready; exit running→terminated. I/O completion normally establishes readiness; policy determines immediate execution. An interrupt can wake a waiter without first running the waiter itself.

Waiting outside memory: a seven-state model



Ready → dispatch → Running; ready-suspended must first be activated

This is a teaching model of whole-process swapping. **Blocked** means the event is incomplete; **suspended** means the process is temporarily ineligible to run in memory. These conditions are independent. Modern virtual memory can reclaim individual pages rather than swapping an entire process.

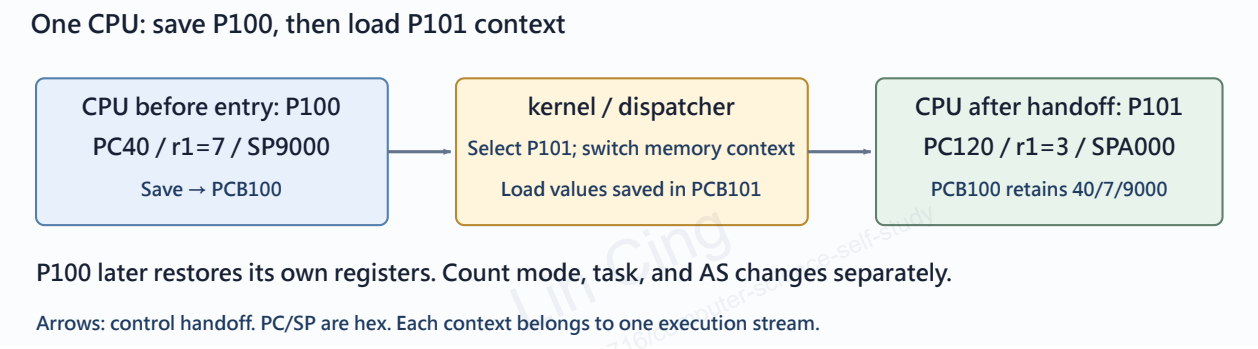
Completion does not mean immediate execution

Time	Event	State of A	Can A receive the CPU?
0	A starts I/O	blocked	No: the event is incomplete
1	Scheduler swaps A out	blocked-suspended	No: incomplete and swapped out
2	I/O completes	ready-suspended	No: it still needs activation
3	A is swapped in	ready	Not yet; awaits selection
4	Dispatcher selects A	running	Yes

Another process may run during times 1–3. If A is swapped in before I/O completes, its path is blocked-suspended → blocked → ready: the same eventual conditions, in the opposite order.

What a context switch preserves

P100 runs PC 0x40, r1=7, SP 0x9000. P101 has saved PC 0x120, r1=3, SP 0xA000. Entry first preserves ISA-required state. The kernel associates the resumable context with P100, then loads P101 and its memory context. Execution resumes at P101's PC, not P100's.



Moment	CPU ownership	Saved/loaded state
Before entry	P100	40/7/9000
Save P100	Kernel	PCB100 retains resumable 40/7/9000
Select/load P101	Kernel→P101	120/3/A000 and correct memory context
Later select P100	P100	Its own context, not P101's registers

What a context switch preserves (continued)

A syscall can change mode without changing process. Switching threads in one process can change registers/stack without user-AS change. Saved PC/restart semantics depend on ISA and event; not every event repeats its instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Queues, scheduler, and dispatcher

Ready queues contain runnable work; device/event queues contain waiters for predicates. Admission controls the resource-constrained job population. The short-term scheduler selects ready execution, and the dispatcher performs context/mode handoff. Long-term admission adjusts multiprogramming; medium-term suspension can remove work temporarily. An OS need not have separate components with all these names.

P1 runs, P2 is ready, and disk R1 is incomplete. P1 submits R1 and waits; P2 runs. Completion makes P1 ready, but a nonpreemptive policy can let P2 continue. Duplicate completion must not duplicate queue membership. A running task must not retain a dispatchable copy that another CPU could run simultaneously.

With 10 ms useful work and 1 ms scheduling/dispatch per cycle, the overhead fraction is 1/(10+1)=1/11. The ratio 1/10 instead measures added cost relative to useful work. This model does not measure cache/TLB disruption.

Lost wakeup: checking again is not enough

Even if a waiter registers, unlocks, and observes WAITING, completion can occur after that observation but before the actual sleep.

Time	Waiter	Completion handler
1	Reads WAITING after unlock	—
2	Has not slept	Sets `done=true` and wakes it
3	Acts on the old decision and sleeps	No further completion

Without coordination with the scheduler, the waiter may sleep forever. The protocol must make release-and-park atomic relative to wakeup, or use an equivalent scheduler-state protocol that preserves an early wakeup.

Connect the predicate to an atomic wait

Protect the predicate and wait queue with the same lock `L`. The waiter acquires `L` and, while `done=false`, registers and calls `park_and_release(L)`; on return it reacquires `L` and checks the predicate again. The completion handler acquires `L`, sets `done=true`, wakes waiters, and releases `L`.

`park_and_release` is an abstract primitive: relative to the handler holding `L`, releasing the lock and committing to wait form an atomic handoff; a wakeup during the handoff must be preserved. It is not ordinary `unlock(); sleep()`. A condition variable with this contract or a kernel wait-queue/scheduler protocol can implement it.

Here the completion handler executes in a context allowed to acquire `L`. A hardware interrupt handler cannot use a sleeping mutex or condition variable; use interrupt-safe locking and wakeup, or defer the work to an appropriate context.

If completion acquires the lock first, the waiter sees `done=true` and does not sleep. Otherwise the handler runs only after registration and the atomic handoff, so it can wake the waiter. Wakeup grants eligibility, not immediate execution. The while loop also handles spurious wakeups or another thread changing the predicate.

CPU-bound and I/O-bound are demand patterns

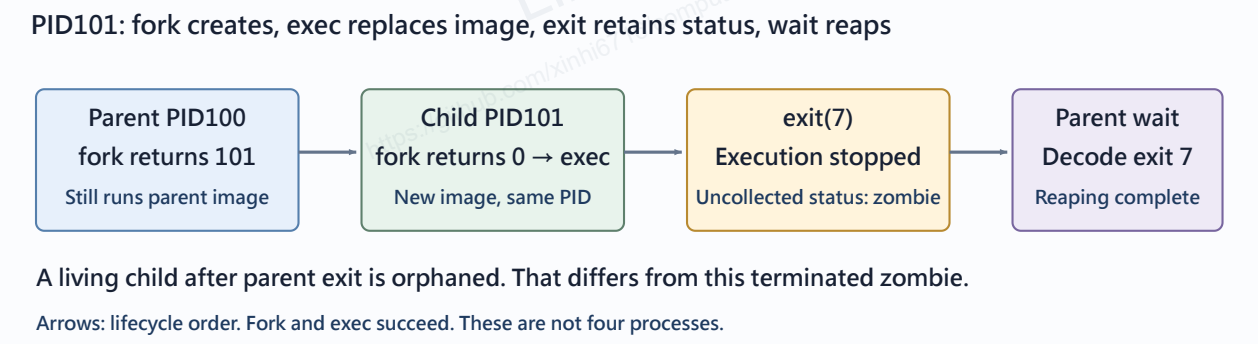
A alternates CPU 1 ms/I/O 9 ms; B needs CPU 10 ms. One CPU can run B during A's independent I/O. Admitting more work whenever CPU utilization is low can overload I/O queues or memory instead. Admission must consider complete demand and queueing.

A larger ready population need not improve throughput. Working sets exceeding frames can cause thrashing and more faults. Suspension/resumption is medium-term resource control, not each CPU selection. Chapter 6 defines waiting, response, and turnaround on common workloads.

Fork, exec, exit, and wait

Fork creates a child with its own task/address-space metadata and required resource references. Private data pages can defer copying with COW. Exec replaces the caller's image; success does not return to its old call site and creates no additional process. Exit ends execution and releases most resources. Wait retrieves child status and completes reaping.

Parent 100 forks 101: parent returns 101, child 0. Child execs worker but remains 101. Worker exits 7; collectible status remains until wait. The parent checks normal exit and decodes 7. Zombie means terminated but unreaped; orphan means the parent relationship has ended, and the child may still run. Reparenting depends on OS, namespace, and subreaper policy. Parent termination can trigger cascading termination or reparenting under the actual system contract.



Two unconditional forks produce four processes

Assume successful forks, no early exit, and every current process executes both unconditional lines. P0 first creates P1. On the second line, P0 creates P2 and P1 creates P3.

Completed point	Executing processes	New children	Alive
Initial	P0	None	1
Fork1	P0	P1	2
Fork2	P0, P1	P2, P3	4

For k such forks, N(k+1)=2N(k), N0=1; induction gives 2^k. Parent-only continuation, child exec/exit, or failure breaks the premises. Only the original parent's second fork gives three total. The tree does not fix PID allocation or print order.

Inherited descriptors can share an open description

Parent fd3 points to O1 at offset 0. Fork copies the fd table entries but can preserve their shared O1 reference. Parent read 100 advances O1 to 100; child read 50 starts at 100 and advances to 150. A separate open of the same inode creates O2 with an independent offset, possibly 0. This differs from private-memory COW.

Fork failure is an error, not the child branch. Exec failure usually returns an error, but after an implementation's point of no return the old image cannot universally be guaranteed intact. Repeated unwaited child exits accumulate status records. General memory/handle release differs from retaining minimal reap metadata.

A multithreaded fork preserves only the calling thread in the child. Copied mutex state can have no surviving owner. Chapter 4 restricts operations before exec; the single-thread flow cannot simply be reused.

Shared memory and message passing

Shared memory establishes common backing/mappings and then exchanges data through loads/stores. It still needs authority and synchronization. Message-passing contracts determine copying, waiting, capacity, and delivery. They do not imply cross-machine execution, zero-copy, or exactly two copies.

Contract	Options to state	Risk if confused
Naming	Process/endpoint or mailbox	PID reuse, rights, cleanup
Synchronization	Each send/receive blocking or nonblocking	Send may await queueing, not processing
Capacity	Rendezvous, bounded, ideal unbounded	Backpressure or finite physical-resource limits
Data boundary	Messages or byte stream	One read need not be one record
Lifetime/failure	Close, unlink, remove, peer crash	EOF, error, and queue removal differ

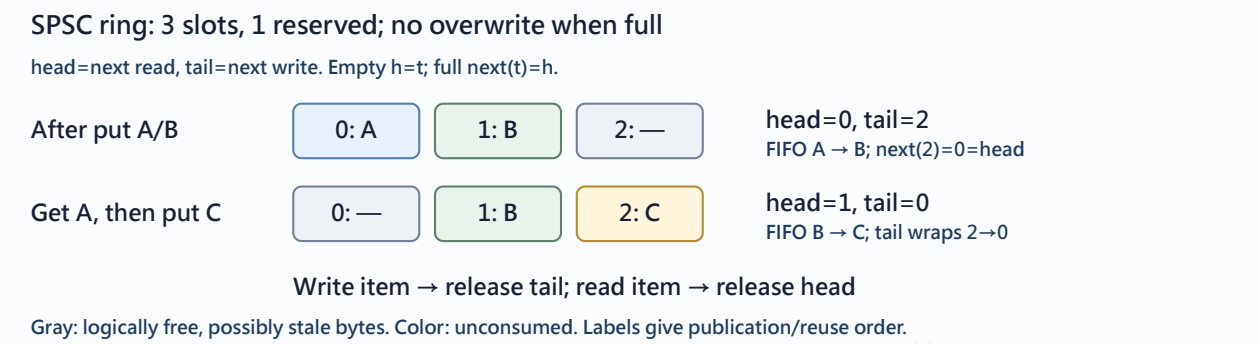
A capacity 2 channel holds complete J1/J2; J3 waits while full. Receiving J1 permits J3. Blocking APIs do not imply zero-capacity rendezvous if send completion means queue insertion.

Three physical ring slots hold two items

One producer and one consumer share slots[3] and atomic head/tail, initially 0. Head is next to consume, tail next to fill. Reserve one slot. Empty means head=tail; full means next(tail)=head; next(x)=(x+1) mod 3.

Event	Head	Tail	Pending FIFO	Result
Initial	0	0	[]	Empty
Put A at 0	0	1	[A]	Publish tail 1
Put B at 1	0	2	[A, B]	Full, next(2)=0=head
Attempt C	0	2	[A, B]	Must not publish overwrite
Get A at 0	1	2	[B]	Release head 1
Put C at 2	1	0	[B, C]	Wrapped, still full
Get B then C	0	0	[]	Empty

Three physical ring slots hold two items (continued)



Publication and reuse need two ordering arguments

Producer acquire-loads head, checks not full, writes slots[tail], then release-stores new tail. Consumer acquire-loads tail, checks not empty, reads slots[head], then release-stores new head. Producer alone writes tail; consumer alone writes head. Local indexes and cross-party loads follow the stated atomic rules. Process-shared atomics require platform support; mapping an arbitrary C++ atomic object across processes is not itself a portable guarantee.

First, slot write precedes release tail, and the consumer's corresponding acquire makes the completed item visible. Second, slot read precedes release head, and the producer's acquire permits reuse only after consumption. Successful put/get increases/decreases logical size by one. Empty/full checks preserve $0 \le \text{size} \le 2$ and FIFO.

Plain shared integers introduce language data races and invalid ordering assumptions. Publishing tail before data permits stale reads. Multiple producers compete for the same tail/slot and violate SPSC. Finite index traces do not replace the general release/acquire argument.

Frame records over a byte stream

Each record has a 2 B big-endian length and payload, max 8 B. Payloads ABC and DE form 00 03 41 42 43 00 02 44 45,9 B. Reads can return [00],[03 41 42],[43 00 02 44 45]. These are not three messages.

Accumulate two header bytes to decode 3, then collect ABC. Decode the next length 2 and collect DE. EOF inside a header or payload is truncation, not an empty completed message. Validate length ≤ 8 and allocation arithmetic before receiving untrusted-sized data. Pipe/TCP ordered streams do not provide application records or atomic remote procedures. Datagram/message truncation and boundaries follow their own APIs.

Shared names, descriptors, and mappings

Creator makes `/course-ring`, sets length 4096 B, and maps MAP_SHARED. Consumer opens/maps the name. P1's base0x100000 and P2's0x700000 can reference the same backing offset 16. Store offsets/indexes, not P1 absolute pointers intended for P2.

Closing a descriptor does not unmap existing mappings. shm_unlink removes the name, preventing new opens through it, while existing references/mappings can survive. MAP_SHARED does not publish completed initialization. A consumer accessing beyond valid object length before sizing is not merely reading an ordinary zero-filled buffer.

Put magic, version, length, and offsets in the region; complete initialization before a synchronized ready notification. Sleeping one second is no substitute for publication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Removal and drain differ

Object	Identifier	Removal	Existing users
POSIX message queue	Name/descriptors	mq_unlink removes name now	Open references survive until final close
System V queue	msqid/key	IPC_RMID removes queue	Blocked send/receive wake with EIDRM
Custom drain channel	Application handle	Close sends, drain, destroy	Only its specified state machine applies

Removal does not universally await consumers draining. Unlink does not automatically send every receiver EOF. Termination and errors require application protocols.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sentinel termination

Use capacity 4 FIFO, one producer/consumer, positive jobs, and 0 reserved as STOP. Send 1,2,3, stop accepting new jobs, then send 0. Queue states are[1]→[1,2]→[1,2,3]→[1,2,3,0]. Consumer completes the jobs before STOP, acknowledges termination, and manager cleans up after acknowledgment. This is not kernel-provided drain.

If 0 is a legal job, use tagged messages. With two consumers, one STOP can leave the other waiting forever; use two sentinels or a closed/broadcast protocol. A crashed consumer can prevent acknowledgment forever, requiring recovery/reassignment policy. One normal trace does not establish exactly-once effects.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mach port and reply rights

In the textbook's historical port/mailbox model, the server has receive right and clients need send rights. Headers specify destination/reply port; typed bodies carry data or controlled rights. Knowing a local port name grants neither global pointer identity nor receive authority.

A sends A1 then A2, and B sends B1. A1, B1, A2 or B1, A1, A2 are allowed, but A2 must not overtake A1 from the same sender under this model. Different senders lack automatic global order. Replying to A1 leaves other requests pending; send success does not equal reply success.

A full queue can wait, time out, or fail under its selected contract. Receiving requires rights. Out-of-line VM/COW can reduce large-data copies without making all inline headers or cross-host messages zero-copy. Rights and shared-memory lifetimes remain protocol obligations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical Windows ALPC connection

A server publishes a connection port. An authorized client handle sends a connection request; acceptance creates private communication endpoints for both directions. Connection setup differs from repeatedly treating each payload as a new connection.

The textbook distinguishes copied small messages from shared-section large data using a historical 256 B threshold. This is not a fixed modern Windows limit. A 128 B request fits the small-message model; a 4096 B payload can use a prepared section with offset/length control. Validate mapping bounds, rights, and writers; a client pointer is untrusted. Applications commonly reach this local mechanism through public RPC interfaces. ALPC, Mach, and POSIX queues share questions of naming, rights, waiting, data, and replies but not interchangeable lifetime contracts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sockets and RPC stubs

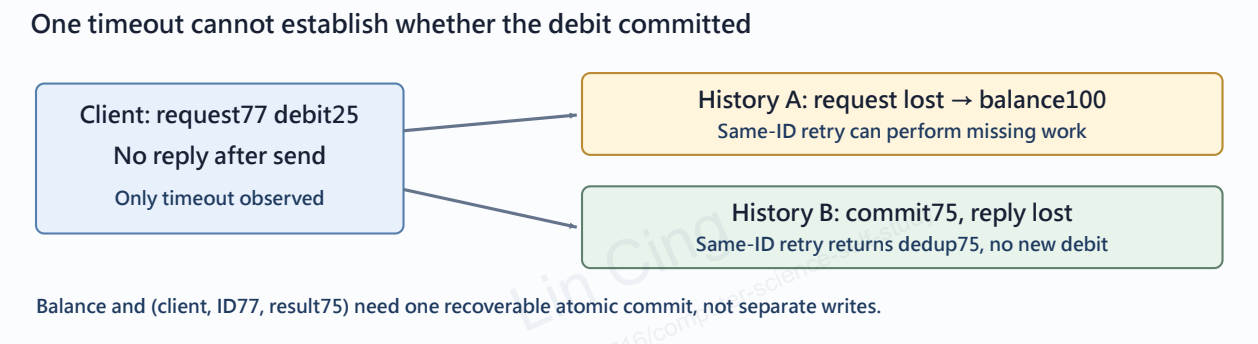
Sockets identify endpoints through address family, local/remote addresses, and ports. A stream server bind/listen/accepts; accept returns a connected endpoint while the listener accepts more. A port is not a PID. Connect success does not establish application-operation completion.

A client stub encodes procedure/request IDs and parameters. A server stub validates/decodes, invokes service, and encodes results. Native endianness, alignment, widths, and pointers are not a wire format. Use id 32/opcode 16/amount 32, all big-endian. ID 77, opcode 1, amount 25 produce 00 00 00 4D | 00 01 | 00 00 00 19,10 B. Validate length/opcode before decoding amount. A transmitted client pointer does not grant memory access.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One timeout, two histories

Balance starts at 100; request 77 debits 25. In history A, the request never arrives and balance stays 100. In B, the server commits 75 but loses the reply. The client sees timeout in both. Retrying with a new ID makes B become 50; not retrying leaves A unperformed.



Deduplicate by authenticated(client identity, request ID). Commit balance and recorded result atomically in one recoverable transaction. The first request 77 changes 100→75 and records 75; duplicate 77 returns 75 without another debit. Concurrent duplicates need a transaction/unique constraint, not independent absent checks followed by two debits.

At-most-once is separate from availability

Dedup records can bound an ID's effects to at most one during retention. Debit then separately record ID leaves a crash window. RAM-only records disappear on restart. Expiring a record while retries remain possible needs an epoch/retention agreement; identities and ID spaces cannot be arbitrarily reused.

At-most-once permits zero execution after permanent loss. At-least-once needs retries and eventual reachability but can repeat non-idempotent effects. Exactly-once requires explicit transaction, durable record, ID scope, failure, and communication assumptions. Reads or setting a value may be idempotent; debit 25 is not.

Anonymous pipes/named FIFOs support local stream inheritance/naming; sockets generalize endpoints across hosts. All still need framing, backpressure, and peer-failure handling. IPC costs also include waking, synchronization, references, and retries.

Execution and data lifetimes

Capacity 2 holds J1/J2. P waits trying J3. C receives J1, frees a slot, and wakes P. P becomes ready and sends J3 after selection. FIFO/capacity remain valid; waking is not immediate execution. Receiving an item is not durable job completion; process exit is not parental reaping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Identities and safety conditions

PID/PCB identifies execution, fd entries reference open descriptions, shared offsets identify data, and request IDs identify retry effects. Equal numbers or paths do not replace references/lifetime proofs.

Correctness requires matching saved/restored execution contexts; one synchronized wait predicate; write-before-publication and read-before-reuse; only complete valid framed records; and joint RPC effect/result commit. Numerical traces expose intermediate states. General invariants explain additional allowed events. Finite testing does not prove every interleaving/crash. New APIs, participant counts, memory orders, or persistence strategies require rechecking their premises.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 3. read(2); pipe(7); msgctl(2); mq_unlink(3).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study